ML ENGINEER GUIDE

AI-Based Monitoring Solution for Oil & Gas Industry

Document Version: 1.0 **Date**: December 2025 **Project Code**: OG-AI-MON-2025 **Audience**: ML Engineers, Data Scientists, AI/ML Researchers

---

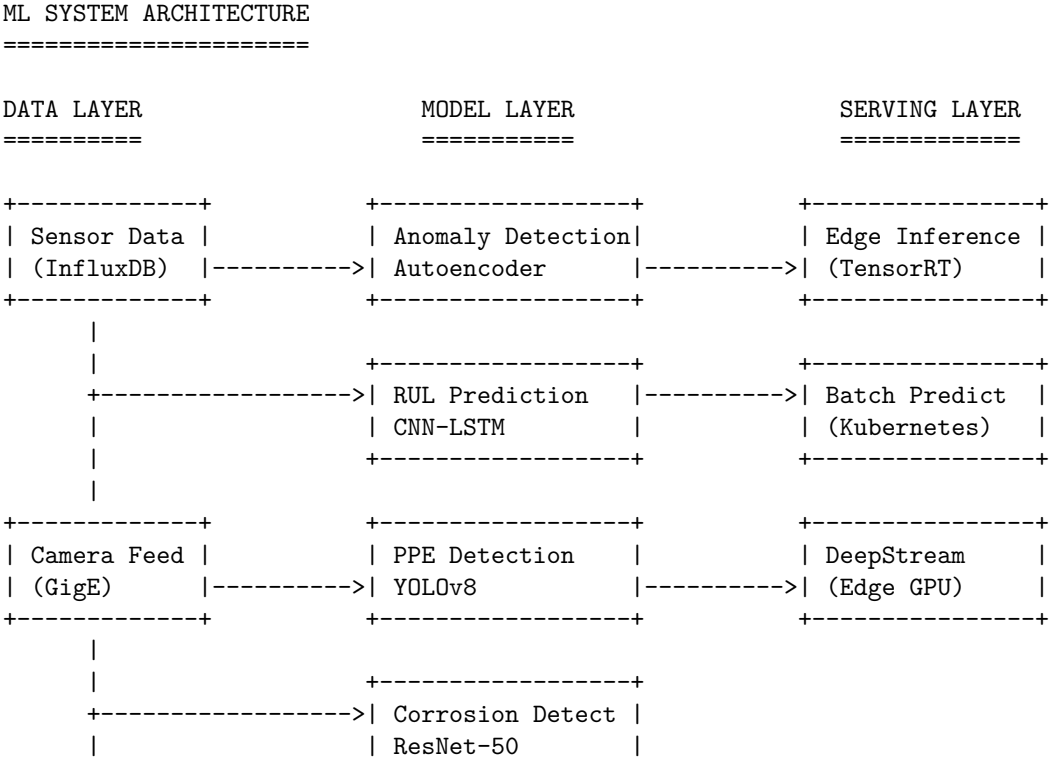
TABLE OF CONTENTS

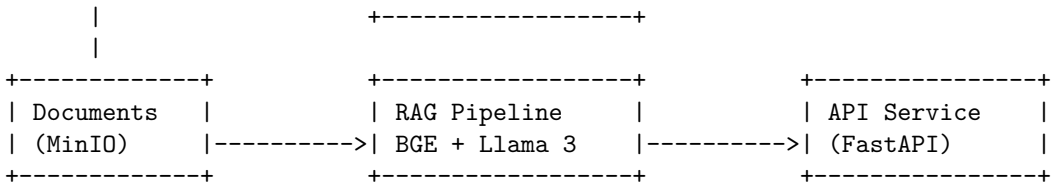
1. ML System Overview
2. Development Environment
3. Data Pipeline
4. Anomaly Detection Model
5. RUL Prediction Model
6. Computer Vision Models
7. RAG System
8. Model Training Infrastructure
9. Model Deployment
10. Monitoring & Retraining
11. Experiment Tracking
12. Best Practices

---

1. ML SYSTEM OVERVIEW

1.1 ML Architecture





1.2 Model Inventory

Model	Type	Framework	Input	Output	Latency Target
Anomaly Detector	Autoencoder	PyTorch	15 features	Anomaly score	<10ms
RUL Predictor	CNN-LSTM	PyTorch	8192 samples	Days remaining	<50ms
PPE Detector	YOLOv8-m	Ultralytics	640x640 RGB	Bounding boxes	<30ms
Corrosion Classifier	ResNet-50	PyTorch	224x224 RGB	4 classes	<20ms
Gas Leak Detector	FLIR AI	Custom	Thermal frame	Leak mask	<50ms
RAG Embedder	BGE-large	Sentence-Transformers	Text	1024d vector	<50ms
RAG Generator	Llama 3.1 70B	vLLM	Prompt	Text	<3s

1.3 Performance Targets

Metric	Target	Current
Anomaly Detection Accuracy	>95%	96.2%
Anomaly False Positive Rate	<5%	3.2%
RUL RMSE	<15 days	12.4 days
PPE Detection mAP@50	>85%	89.2%
Corrosion Classification	>90%	91.5%
RAG Relevance Score	>0.85	0.87
Edge Inference Latency	<100ms	45ms

2. DEVELOPMENT ENVIRONMENT

2.1 Hardware Requirements

Component	Development	Training	Inference
CPU	8+ cores	32+ cores	4+ cores
RAM	32GB	128GB	16GB
GPU	RTX 3080	A100 80GB	Jetson AGX
Storage	500GB SSD	2TB NVMe	256GB NVMe

2.2 Software Setup

```
# Clone ML repository
git clone git@github.com:oilgas/ml-models.git
cd ml-models

# Create conda environment
conda create -n oilgas-ml python=3.11
conda activate oilgas-ml

# Install PyTorch with CUDA
pip install torch==2.1.0 torchvision==0.16.0 torchaudio==2.1.0 --index-url https://download.pytorch.org/

# Install ML dependencies
pip install -r requirements.txt

# Install development tools
pip install -r requirements-dev.txt

# Verify GPU
python -c "import torch; print(f'CUDA available: {torch.cuda.is_available()}')"
python -c "import torch; print(f'GPU: {torch.cuda.get_device_name(0)}')"
```

requirements.txt

```
# Core ML
torch==2.1.0
torchvision==0.16.0
numpy==1.26.0
pandas==2.1.0
scikit-learn==1.3.0

# Deep Learning
pytorch-lightning==2.1.0
transformers==4.35.0
sentence-transformers==2.2.2

# Computer Vision
ultralytics==8.0.200
opencv-python==4.8.1.78
albumentations==1.3.1

# Time Series
tslearn==0.6.2
tsfresh==0.20.1

# Experiment Tracking
mlflow==2.8.0
wandb==0.16.0

# Data Processing
pyarrow==14.0.0
influxdb-client==1.38.0
pymilvus==2.3.0
```

```
# Visualization
matplotlib==3.8.0
seaborn==0.13.0
plotly==5.18.0

# Utilities
pyyaml==6.0
loguru==0.7.2
tqdm==4.66.0
```

2.3 Data Access

```python
# config/data_config.py
from dataclasses import dataclass

@dataclass
class DataConfig:
    # InfluxDB (time-series data)
    influxdb_url: str = "http://influxdb.oilgas.internal:8086"
    influxdb_org: str = "oilgas"
    influxdb_token: str = "${INFLUXDB_TOKEN}"
    sensor_bucket: str = "sensor_data"
    features_bucket: str = "features"

    # PostgreSQL (metadata)
    postgres_host: str = "postgres.oilgas.internal"
    postgres_port: int = 5432
    postgres_db: str = "oilgas_ai"
    postgres_user: str = "ml_readonly"

    # MinIO (raw files, models)
    minio_endpoint: str = "minio.oilgas.internal:9000"
    minio_bucket: str = "ml-data"

    # MLflow
    mlflow_tracking_uri: str = "http://mlflow.oilgas.internal:5000"
```

```python
# utils/data_loader.py
from influxdb_client import InfluxDBClient
import pandas as pd

class SensorDataLoader:
    def __init__(self, config: DataConfig):
        self.client = InfluxDBClient(
            url=config.influxdb_url,
            token=config.influxdb_token,
            org=config.influxdb_org
        )
        self.query_api = self.client.query_api()

    def load_vibration_features(
        self,
        asset_id: str,
        start_time: str,
        end_time: str
```

```
    ) -> pd.DataFrame:
        """Load vibration features for training"""
        query = f'''
            from(bucket: "features")
            |> range(start: {start_time}, stop: {end_time})
            |> filter(fn: (r) => r._measurement == "vibration_features")
            |> filter(fn: (r) => r.asset_id == "{asset_id}")
            |> pivot(rowKey:["_time"], columnKey: ["_field"], valueColumn: "_value")
        '''
        result = self.query_api.query_data_frame(query)
        return result

    def load_training_dataset(
        self,
        asset_ids: list,
        start_time: str,
        end_time: str
    ) -> pd.DataFrame:
        """Load training dataset for multiple assets"""
        dfs = []
        for asset_id in asset_ids:
            df = self.load_vibration_features(asset_id, start_time, end_time)
            df['asset_id'] = asset_id
            dfs.append(df)
        return pd.concat(dfs, ignore_index=True)
```

---

3. DATA PIPELINE

3.1 Feature Engineering Pipeline

```
# features/vibration_features.py
import numpy as np
from scipy import signal, stats
from scipy.fft import rfft, rfftfreq
from dataclasses import dataclass
from typing import Tuple
import torch

@dataclass
class BearingParams:
    """Bearing geometry parameters"""
    n_balls: int
    ball_diameter: float
    pitch_diameter: float
    contact_angle: float  # degrees

    def get_fault_frequencies(self, rpm: float) -> dict:
        """Calculate bearing fault frequencies"""
        fr = rpm / 60  # Rotational frequency
        bd = self.ball_diameter
        pd = self.pitch_diameter
        nb = self.n_balls
```

```
        phi = np.radians(self.contact_angle)

        return {
            'bpfo': (nb / 2) * fr * (1 - (bd / pd) * np.cos(phi)),
            'bpfi': (nb / 2) * fr * (1 + (bd / pd) * np.cos(phi)),
            'bsf': (pd / (2 * bd)) * fr * (1 - ((bd / pd) * np.cos(phi))**2),
            'ftf': 0.5 * fr * (1 - (bd / pd) * np.cos(phi))
        }


class VibrationFeatureExtractor:
    """Extract vibration features from raw waveform"""

    def __init__(
        self,
        sampling_rate: float = 25600,
        bearing_params: BearingParams = None
    ):
        self.fs = sampling_rate
        self.bearing = bearing_params or BearingParams(
            n_balls=9, ball_diameter=7.94, pitch_diameter=39.04, contact_angle=0
        )

    def extract_time_domain(self, waveform: np.ndarray) -> dict:
        """Extract time-domain features"""
        # Remove DC offset
        x = waveform - np.mean(waveform)

        return {
            'rms': np.sqrt(np.mean(x**2)),
            'peak': np.max(np.abs(x)),
            'peak_to_peak': np.max(x) - np.min(x),
            'crest_factor': np.max(np.abs(x)) / np.sqrt(np.mean(x**2)),
            'kurtosis': stats.kurtosis(x),
            'skewness': stats.skew(x),
            'std': np.std(x),
            'variance': np.var(x),
            'shape_factor': np.sqrt(np.mean(x**2)) / np.mean(np.abs(x)),
            'impulse_factor': np.max(np.abs(x)) / np.mean(np.abs(x)),
            'clearance_factor': np.max(np.abs(x)) / (np.mean(np.sqrt(np.abs(x))))**2
        }

    def extract_frequency_domain(
        self,
        waveform: np.ndarray,
        rpm: float
    ) -> dict:
        """Extract frequency-domain features"""
        x = waveform - np.mean(waveform)
        n = len(x)

        # FFT
        fft_vals = np.abs(rfft(x)) / n
        freqs = rfftfreq(n, 1 / self.fs)
```

```
        # Power spectrum
        psd = fft_vals**2

        # Rotational frequency
        fr = rpm / 60

        def get_amplitude(target_freq: float, bandwidth: float = 5) -> float:
            mask = (freqs >= target_freq - bandwidth) & (freqs <= target_freq + bandwidth)
            return float(np.max(fft_vals[mask])) if np.any(mask) else 0.0

        # Harmonic amplitudes
        features = {
            'fft_1x': get_amplitude(fr),
            'fft_2x': get_amplitude(2 * fr),
            'fft_3x': get_amplitude(3 * fr),
            'fft_4x': get_amplitude(4 * fr),
        }

        # Bearing fault frequencies
        fault_freqs = self.bearing.get_fault_frequencies(rpm)
        for name, freq in fault_freqs.items():
            features[name] = get_amplitude(freq)
            features[f'{name}_2x'] = get_amplitude(2 * freq)

        # Spectral statistics
        features['spectral_centroid'] = np.sum(freqs * psd) / np.sum(psd)
        features['spectral_spread'] = np.sqrt(
            np.sum(((freqs - features['spectral_centroid'])**2) * psd) / np.sum(psd)
        )
        features['spectral_entropy'] = stats.entropy(psd / np.sum(psd))

        return features

    def extract_envelope(self, waveform: np.ndarray) -> dict:
        """Extract envelope (demodulation) features"""
        x = waveform - np.mean(waveform)

        # Hilbert transform for envelope
        analytic = signal.hilbert(x)
        envelope = np.abs(analytic)

        return {
            'envelope_rms': np.sqrt(np.mean(envelope**2)),
            'envelope_peak': np.max(envelope),
            'envelope_kurtosis': stats.kurtosis(envelope),
            'envelope_skewness': stats.skew(envelope)
        }

    def extract_all(
        self,
        waveform: np.ndarray,
        rpm: float
    ) -> dict:
```

```
        """Extract all features"""
        features = {}
        features.update(self.extract_time_domain(waveform))
        features.update(self.extract_frequency_domain(waveform, rpm))
        features.update(self.extract_envelope(waveform))
        return features

    def extract_batch(
        self,
        waveforms: np.ndarray,  # Shape: (batch, samples)
        rpms: np.ndarray
    ) -> np.ndarray:
        """Extract features for batch of waveforms"""
        features_list = []
        for i in range(len(waveforms)):
            features = self.extract_all(waveforms[i], rpms[i])
            features_list.append(list(features.values()))
        return np.array(features_list)
```

3.2 Data Preprocessing

```
# preprocessing/pipeline.py
import numpy as np
from sklearn.preprocessing import StandardScaler, RobustScaler
from sklearn.impute import KNNImputer
from scipy import signal
import pickle
from pathlib import Path

class PreprocessingPipeline:
    """Preprocessing pipeline for vibration data"""

    def __init__(self, config: dict):
        self.config = config
        self.scaler = None
        self.imputer = None

    def denoise(self, waveform: np.ndarray) -> np.ndarray:
        """Apply denoising filter"""
        # Butterworth low-pass filter
        fc = self.config.get('cutoff_freq', 10000)  # 10 kHz
        fs = self.config.get('sampling_rate', 25600)
        order = self.config.get('filter_order', 4)

        b, a = signal.butter(order, fc / (fs / 2), btype='low')
        return signal.filtfilt(b, a, waveform)

    def remove_outliers(self, features: np.ndarray) -> np.ndarray:
        """Remove outliers using IQR method"""
        q1 = np.percentile(features, 25, axis=0)
        q3 = np.percentile(features, 75, axis=0)
        iqr = q3 - q1

        lower = q1 - 1.5 * iqr
```

```
        upper = q3 + 1.5 * iqr

        # Clip outliers
        return np.clip(features, lower, upper)

    def impute_missing(self, features: np.ndarray) -> np.ndarray:
        """Impute missing values using KNN"""
        if self.imputer is None:
            self.imputer = KNNImputer(n_neighbors=5)
            return self.imputer.fit_transform(features)
        return self.imputer.transform(features)

    def normalize(
        self,
        features: np.ndarray,
        fit: bool = False
    ) -> np.ndarray:
        """Normalize features"""
        if fit or self.scaler is None:
            self.scaler = RobustScaler()
            return self.scaler.fit_transform(features)
        return self.scaler.transform(features)

    def fit_transform(self, features: np.ndarray) -> np.ndarray:
        """Full preprocessing pipeline (fit)"""
        features = self.remove_outliers(features)
        features = self.impute_missing(features)
        features = self.normalize(features, fit=True)
        return features

    def transform(self, features: np.ndarray) -> np.ndarray:
        """Full preprocessing pipeline (transform only)"""
        features = self.remove_outliers(features)
        features = self.impute_missing(features)
        features = self.normalize(features, fit=False)
        return features

    def save(self, path: Path):
        """Save preprocessing artifacts"""
        artifacts = {
            'scaler': self.scaler,
            'imputer': self.imputer,
            'config': self.config
        }
        with open(path / 'preprocessing.pkl', 'wb') as f:
            pickle.dump(artifacts, f)

    @classmethod
    def load(cls, path: Path) -> 'PreprocessingPipeline':
        """Load preprocessing artifacts"""
        with open(path / 'preprocessing.pkl', 'rb') as f:
            artifacts = pickle.load(f)
        pipeline = cls(artifacts['config'])
        pipeline.scaler = artifacts['scaler']
```

```
        pipeline.imputer = artifacts['imputer']
        return pipeline
```

---

4. ANOMALY DETECTION MODEL

4.1 Model Architecture

```
# models/anomaly_detection/model.py
import torch
import torch.nn as nn
from typing import Tuple

class AnomalyAutoencoder(nn.Module):
    """
    Autoencoder for vibration anomaly detection.

    Architecture:
    - Encoder: 15 -> 10 -> 5 -> 3 (latent)
    - Decoder: 3 -> 5 -> 10 -> 15

    Anomaly score = reconstruction error (MSE)
    """

    def __init__(
        self,
        input_dim: int = 15,
        latent_dim: int = 3,
        hidden_dims: list = [10, 5],
        dropout: float = 0.1
    ):
        super().__init__()

        self.input_dim = input_dim
        self.latent_dim = latent_dim

        # Encoder
        encoder_layers = []
        prev_dim = input_dim
        for hidden_dim in hidden_dims:
            encoder_layers.extend([
                nn.Linear(prev_dim, hidden_dim),
                nn.BatchNorm1d(hidden_dim),
                nn.LeakyReLU(0.2),
                nn.Dropout(dropout)
            ])
            prev_dim = hidden_dim

        encoder_layers.append(nn.Linear(prev_dim, latent_dim))
        self.encoder = nn.Sequential(*encoder_layers)

        # Decoder
        decoder_layers = []
```

```
        prev_dim = latent_dim
        for hidden_dim in reversed(hidden_dims):
            decoder_layers.extend([
                nn.Linear(prev_dim, hidden_dim),
                nn.BatchNorm1d(hidden_dim),
                nn.LeakyReLU(0.2),
                nn.Dropout(dropout)
            ])
            prev_dim = hidden_dim

        decoder_layers.append(nn.Linear(prev_dim, input_dim))
        self.decoder = nn.Sequential(*decoder_layers)

        # Initialize weights
        self._init_weights()

    def _init_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.kaiming_normal_(m.weight, mode='fan_in', nonlinearity='leaky_relu')
                if m.bias is not None:
                    nn.init.zeros_(m.bias)

    def encode(self, x: torch.Tensor) -> torch.Tensor:
        """Encode input to latent space"""
        return self.encoder(x)

    def decode(self, z: torch.Tensor) -> torch.Tensor:
        """Decode from latent space"""
        return self.decoder(z)

    def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        """Forward pass"""
        z = self.encode(x)
        x_recon = self.decode(z)
        return x_recon, z

    def anomaly_score(self, x: torch.Tensor) -> torch.Tensor:
        """Calculate anomaly score (reconstruction error)"""
        x_recon, _ = self.forward(x)
        return torch.mean((x - x_recon)**2, dim=1)

    def predict(
        self,
        x: torch.Tensor,
        threshold: float = None
    ) -> Tuple[torch.Tensor, torch.Tensor]:
        """
        Predict anomaly.

        Returns:
            scores: Anomaly scores
            labels: Binary labels (1=anomaly) if threshold provided
        """
```

```
        scores = self.anomaly_score(x)
        if threshold is not None:
            labels = (scores > threshold).long()
            return scores, labels
        return scores, None
```

4.2 Training Script

```
# models/anomaly_detection/train.py
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
import mlflow
import mlflow.pytorch
from loguru import logger
from pathlib import Path
import yaml
from datetime import datetime

from model import AnomalyAutoencoder
from ...preprocessing.pipeline import PreprocessingPipeline
from ...utils.data_loader import SensorDataLoader

def train_anomaly_detector(config_path: str):
    """Train anomaly detection model"""

    # Load config
    with open(config_path) as f:
        config = yaml.safe_load(f)

    # Setup MLflow
    mlflow.set_tracking_uri(config['mlflow']['tracking_uri'])
    mlflow.set_experiment(config['mlflow']['experiment_name'])

    with mlflow.start_run(run_name=f"train_{datetime.now().strftime('%Y%m%d_%H%M')}"):
        # Log config
        mlflow.log_params(config['model'])
        mlflow.log_params(config['training'])

        # Load data
        logger.info("Loading training data...")
        data_loader = SensorDataLoader(config['data'])
        df = data_loader.load_training_dataset(
            asset_ids=config['data']['training_assets'],
            start_time=config['data']['start_time'],
            end_time=config['data']['end_time']
        )

        # Extract feature columns
        feature_cols = [c for c in df.columns if c not in ['_time', 'asset_id', 'label']]
        X = df[feature_cols].values

        # Preprocessing
```

```
logger.info("Preprocessing data...")
preprocessor = PreprocessingPipeline(config['preprocessing'])
X = preprocessor.fit_transform(X)

# Train/val split
n_train = int(len(X) * 0.8)
X_train, X_val = X[:n_train], X[n_train:]

# Create dataloaders
train_dataset = TensorDataset(torch.FloatTensor(X_train))
val_dataset = TensorDataset(torch.FloatTensor(X_val))

train_loader = DataLoader(
    train_dataset,
    batch_size=config['training']['batch_size'],
    shuffle=True,
    num_workers=4
)
val_loader = DataLoader(
    val_dataset,
    batch_size=config['training']['batch_size']
)

# Initialize model
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
logger.info(f"Using device: {device}")

model = AnomalyAutoencoder(
    input_dim=len(feature_cols),
    latent_dim=config['model']['latent_dim'],
    hidden_dims=config['model']['hidden_dims'],
    dropout=config['model']['dropout']
).to(device)

# Loss and optimizer
criterion = nn.MSELoss()
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=config['training']['learning_rate'],
    weight_decay=config['training']['weight_decay']
)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', patience=5, factor=0.5
)

# Training loop
best_val_loss = float('inf')
patience_counter = 0
reconstruction_errors = []

for epoch in range(config['training']['epochs']):
    # Train
    model.train()
    train_loss = 0
```

```
    for batch in train_loader:
        x = batch[0].to(device)
        optimizer.zero_grad()

        x_recon, _ = model(x)
        loss = criterion(x_recon, x)

        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

        train_loss += loss.item()

    train_loss /= len(train_loader)

    # Validate
    model.eval()
    val_loss = 0
    val_errors = []

    with torch.no_grad():
        for batch in val_loader:
            x = batch[0].to(device)
            x_recon, _ = model(x)
            loss = criterion(x_recon, x)
            val_loss += loss.item()

            # Per-sample error
            errors = torch.mean((x - x_recon)**2, dim=1)
            val_errors.extend(errors.cpu().numpy())

    val_loss /= len(val_loader)
    scheduler.step(val_loss)

    # Log metrics
    mlflow.log_metrics({
        'train_loss': train_loss,
        'val_loss': val_loss,
        'learning_rate': optimizer.param_groups[0]['lr']
    }, step=epoch)

    logger.info(f"Epoch {epoch+1}: train_loss={train_loss:.6f}, val_loss={val_loss:.6f}")

    # Early stopping
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        patience_counter = 0
        reconstruction_errors = val_errors
        torch.save(model.state_dict(), 'best_model.pt')
    else:
        patience_counter += 1
        if patience_counter >= config['training']['early_stopping_patience']:
            logger.info(f"Early stopping at epoch {epoch+1}")
            break
```

```
        # Calculate threshold
        errors = np.array(reconstruction_errors)
        threshold_config = {
            'mean': float(np.mean(errors)),
            'std': float(np.std(errors)),
            'threshold_2sigma': float(np.mean(errors) + 2 * np.std(errors)),
            'threshold_3sigma': float(np.mean(errors) + 3 * np.std(errors)),
            'threshold_percentile_99': float(np.percentile(errors, 99))
        }
        mlflow.log_dict(threshold_config, 'threshold_config.json')

        # Save preprocessing
        output_dir = Path(config['output']['dir'])
        output_dir.mkdir(parents=True, exist_ok=True)
        preprocessor.save(output_dir)

        # Export model
        model.load_state_dict(torch.load('best_model.pt'))
        model.eval()

        # ONNX export
        dummy_input = torch.randn(1, len(feature_cols)).to(device)
        torch.onnx.export(
            model,
            dummy_input,
            output_dir / 'model.onnx',
            input_names=['input'],
            output_names=['reconstructed', 'latent'],
            dynamic_axes={'input': {0: 'batch_size'}}
        )

        # TorchScript export
        scripted = torch.jit.script(model)
        scripted.save(output_dir / 'model.pt')

        # Log artifacts
        mlflow.log_artifacts(str(output_dir))

        # Register model
        mlflow.pytorch.log_model(
            model,
            "model",
            registered_model_name=config['mlflow']['model_name']
        )

        logger.info("Training complete!")
        logger.info(f"Best validation loss: {best_val_loss:.6f}")
        logger.info(f"Threshold (3-sigma): {threshold_config['threshold_3sigma']:.6f}")

if __name__ == '__main__':
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument('--config', required=True)
```

```
    args = parser.parse_args()
    train_anomaly_detector(args.config)
```

---

5. RUL PREDICTION MODEL

5.1 Model Architecture

```python
# models/rul_prediction/model.py
import torch
import torch.nn as nn
import torch.nn.functional as F
from typing import Tuple

class CNNLSTM_RUL(nn.Module):
    """
    CNN-LSTM model for Remaining Useful Life prediction.

    Architecture:
    1. 1D CNN for local pattern extraction
    2. LSTM for temporal dependencies
    3. Attention mechanism for important timesteps
    4. Fully connected for RUL output

    Input: (batch, sequence_length, features)
    Output: RUL in days with uncertainty
    """

    def __init__(
        self,
        input_features: int = 15,
        sequence_length: int = 50,
        cnn_channels: list = [32, 64, 128],
        lstm_hidden: int = 128,
        lstm_layers: int = 2,
        attention_heads: int = 4,
        dropout: float = 0.3,
        mc_dropout: bool = True
    ):
        super().__init__()

        self.mc_dropout = mc_dropout

        # CNN layers
        cnn_layers = []
        in_channels = input_features
        for out_channels in cnn_channels:
            cnn_layers.extend([
                nn.Conv1d(in_channels, out_channels, kernel_size=3, padding=1),
                nn.BatchNorm1d(out_channels),
                nn.ReLU(),
                nn.MaxPool1d(2),
                nn.Dropout(dropout)
```

```
            ])
            in_channels = out_channels
        self.cnn = nn.Sequential(*cnn_layers)

        # Calculate CNN output size
        cnn_output_length = sequence_length // (2 ** len(cnn_channels))
        cnn_output_features = cnn_channels[-1]

        # LSTM
        self.lstm = nn.LSTM(
            input_size=cnn_output_features,
            hidden_size=lstm_hidden,
            num_layers=lstm_layers,
            batch_first=True,
            dropout=dropout if lstm_layers > 1 else 0,
            bidirectional=True
        )

        # Attention
        self.attention = nn.MultiheadAttention(
            embed_dim=lstm_hidden * 2,  # bidirectional
            num_heads=attention_heads,
            dropout=dropout,
            batch_first=True
        )

        # Output layers
        self.fc = nn.Sequential(
            nn.Linear(lstm_hidden * 2, 64),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(32, 2)  # mean and log_variance
        )

        self._init_weights()

    def _init_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Conv1d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
            elif isinstance(m, nn.Linear):
                nn.init.xavier_normal_(m.weight)
                if m.bias is not None:
                    nn.init.zeros_(m.bias)

    def forward(
        self,
        x: torch.Tensor
    ) -> Tuple[torch.Tensor, torch.Tensor]:
        """
        Forward pass.
```

```
        Args:
            x: Input tensor (batch, sequence, features)

        Returns:
            rul_mean: Predicted RUL
            rul_std: Uncertainty (standard deviation)
        """
        batch_size = x.size(0)

        # CNN expects (batch, channels, length)
        x = x.permute(0, 2, 1)
        x = self.cnn(x)
        x = x.permute(0, 2, 1)  # Back to (batch, length, features)

        # LSTM
        lstm_out, _ = self.lstm(x)

        # Self-attention
        attn_out, _ = self.attention(lstm_out, lstm_out, lstm_out)

        # Global average pooling
        context = torch.mean(attn_out, dim=1)

        # Output
        output = self.fc(context)
        rul_mean = F.softplus(output[:, 0])  # RUL must be positive
        rul_log_var = output[:, 1]
        rul_std = torch.exp(0.5 * rul_log_var)

        return rul_mean, rul_std

    def predict_with_uncertainty(
        self,
        x: torch.Tensor,
        n_samples: int = 100
    ) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
        """
        Predict with Monte Carlo Dropout uncertainty.

        Returns:
            mean: Mean prediction
            std: Total uncertainty
            epistemic: Model uncertainty
        """
        if not self.mc_dropout:
            rul_mean, rul_std = self.forward(x)
            return rul_mean, rul_std, torch.zeros_like(rul_std)

        # Enable dropout at inference
        self.train()
        predictions = []

        with torch.no_grad():
```

```
            for _ in range(n_samples):
                rul_mean, _ = self.forward(x)
                predictions.append(rul_mean)

        predictions = torch.stack(predictions)

        mean = predictions.mean(dim=0)
        epistemic = predictions.std(dim=0)

        # Get aleatoric uncertainty
        self.eval()
        _, aleatoric = self.forward(x)

        total_std = torch.sqrt(epistemic**2 + aleatoric**2)

        return mean, total_std, epistemic
```

5.2 Custom Loss Function

```
# models/rul_prediction/loss.py
import torch
import torch.nn as nn

class RULLoss(nn.Module):
    """
    Custom loss for RUL prediction with uncertainty.

    Combines:
    1. Negative log-likelihood for Gaussian output
    2. Asymmetric penalty (late predictions more costly)
    """

    def __init__(
        self,
        asymmetry_factor: float = 2.0,
        uncertainty_weight: float = 0.1
    ):
        super().__init__()
        self.asymmetry = asymmetry_factor
        self.uncertainty_weight = uncertainty_weight

    def forward(
        self,
        pred_mean: torch.Tensor,
        pred_std: torch.Tensor,
        target: torch.Tensor
    ) -> torch.Tensor:
        # Gaussian NLL
        variance = pred_std**2
        nll = 0.5 * (torch.log(variance) + (target - pred_mean)**2 / variance)

        # Asymmetric penalty
        error = target - pred_mean
        asymmetric_weight = torch.where(
```

```
            error > 0,  # Predicted < actual (early prediction)
            torch.ones_like(error),
            torch.ones_like(error) * self.asymmetry  # Late prediction penalty
        )
        asymmetric_loss = asymmetric_weight * error**2

        # Uncertainty regularization (prevent predicting infinite uncertainty)
        uncertainty_reg = self.uncertainty_weight * torch.mean(pred_std)

        return torch.mean(nll + asymmetric_loss) + uncertainty_reg
```

---

6. COMPUTER VISION MODELS

6.1 PPE Detection

```
# models/computer_vision/ppe_detection.py
from ultralytics import YOLO
import torch
import numpy as np
from pathlib import Path

class PPEDetector:
    """
    PPE (Personal Protective Equipment) detector using YOLOv8.

    Detects:
    - Hard hat
    - Safety vest
    - Safety glasses
    - Gloves
    - Safety boots
    """

    CLASSES = [
        'hard_hat', 'no_hard_hat',
        'vest', 'no_vest',
        'glasses', 'no_glasses',
        'gloves', 'no_gloves'
    ]

    def __init__(self, model_path: str = None):
        if model_path:
            self.model = YOLO(model_path)
        else:
            self.model = YOLO('yolov8m.pt')  # Base model

    def train(
        self,
        data_yaml: str,
        epochs: int = 100,
        imgsz: int = 640,
        batch: int = 16
```

```
):
    """Train PPE detection model"""
    results = self.model.train(
        data=data_yaml,
        epochs=epochs,
        imgsz=imgsz,
        batch=batch,
        device=0,
        project='runs/ppe_detection',
        name='train',
        # Augmentation
        augment=True,
        hsv_h=0.015,
        hsv_s=0.7,
        hsv_v=0.4,
        degrees=10,
        translate=0.1,
        scale=0.5,
        fliplr=0.5,
        mosaic=1.0,
        mixup=0.1,
        # Training params
        optimizer='AdamW',
        lr0=0.01,
        lrf=0.01,
        momentum=0.937,
        weight_decay=0.0005,
        warmup_epochs=3,
        # Callbacks
        save=True,
        save_period=10,
        val=True
    )
    return results

def predict(
    self,
    image: np.ndarray,
    conf_threshold: float = 0.5
) -> dict:
    """
    Predict PPE in image.

    Returns:
        {
            'detections': [{'class': 'hard_hat', 'confidence': 0.95, 'bbox': [x1,y1,x2,y2]}],
            'compliance': {'hard_hat': True, 'vest': False, ...},
            'compliance_score': 0.75
        }
    """
    results = self.model.predict(image, conf=conf_threshold, verbose=False)[0]

    detections = []
    compliance = {
```

```
            'hard_hat': False,
            'vest': False,
            'glasses': False,
            'gloves': False
        }

        for box in results.boxes:
            cls_id = int(box.cls[0])
            cls_name = self.CLASSES[cls_id]
            conf = float(box.conf[0])
            bbox = box.xyxy[0].tolist()

            detections.append({
                'class': cls_name,
                'confidence': conf,
                'bbox': bbox
            })

            # Update compliance
            if 'no_' not in cls_name:
                compliance[cls_name] = True

        compliance_score = sum(compliance.values()) / len(compliance)

        return {
            'detections': detections,
            'compliance': compliance,
            'compliance_score': compliance_score
        }

    def export_tensorrt(self, output_path: str):
        """Export to TensorRT for edge deployment"""
        self.model.export(
            format='engine',
            device=0,
            half=True,
            imgsz=640,
            batch=1,
            workspace=4
        )
```

6.2 Corrosion Detection

```
# models/computer_vision/corrosion_detection.py
import torch
import torch.nn as nn
import torchvision.models as models
from torchvision import transforms
import numpy as np
from PIL import Image
from enum import IntEnum

class CorrosionLevel(IntEnum):
    NONE = 0
```

```
    LIGHT = 1
    MODERATE = 2
    SEVERE = 3

class CorrosionClassifier:
    """
    Corrosion severity classifier using ResNet-50.

    Classes:
    - 0: No corrosion
    - 1: Light corrosion
    - 2: Moderate corrosion
    - 3: Severe corrosion
    """

    def __init__(self, model_path: str = None, device: str = 'cuda'):
        self.device = torch.device(device if torch.cuda.is_available() else 'cpu')

        # Load model
        self.model = models.resnet50(pretrained=False)
        self.model.fc = nn.Sequential(
            nn.Dropout(0.3),
            nn.Linear(2048, 512),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(512, 4)
        )

        if model_path:
            self.model.load_state_dict(torch.load(model_path, map_location=self.device))

        self.model.to(self.device)
        self.model.eval()

        # Preprocessing
        self.transform = transforms.Compose([
            transforms.Resize(256),
            transforms.CenterCrop(224),
            transforms.ToTensor(),
            transforms.Normalize(
                mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225]
            )
        ])

    def predict(self, image: np.ndarray) -> dict:
        """
        Classify corrosion severity.

        Returns:
            {
                'level': CorrosionLevel.MODERATE,
                'level_name': 'moderate',
                'confidence': 0.87,
```

```
            'probabilities': [0.02, 0.08, 0.87, 0.03]
        }
    """
    # Convert to PIL
    if isinstance(image, np.ndarray):
        image = Image.fromarray(image)

    # Preprocess
    x = self.transform(image).unsqueeze(0).to(self.device)

    # Predict
    with torch.no_grad():
        logits = self.model(x)
        probs = torch.softmax(logits, dim=1)[0]
        pred_class = torch.argmax(probs).item()
        confidence = probs[pred_class].item()

    return {
        'level': CorrosionLevel(pred_class),
        'level_name': CorrosionLevel(pred_class).name.lower(),
        'confidence': confidence,
        'probabilities': probs.cpu().numpy().tolist()
    }

@staticmethod
def train_model(
    train_dir: str,
    val_dir: str,
    epochs: int = 50,
    batch_size: int = 32,
    learning_rate: float = 1e-4
):
    """Train corrosion classifier"""
    from torchvision.datasets import ImageFolder
    from torch.utils.data import DataLoader
    import pytorch_lightning as pl

    # Data transforms with augmentation
    train_transform = transforms.Compose([
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.RandomVerticalFlip(),
        transforms.RandomRotation(30),
        transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])

    val_transform = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])
```

```
# Datasets
train_dataset = ImageFolder(train_dir, transform=train_transform)
val_dataset = ImageFolder(val_dir, transform=val_transform)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=4)
val_loader = DataLoader(val_dataset, batch_size=batch_size, num_workers=4)

# Model
model = models.resnet50(pretrained=True)
for param in model.parameters():
    param.requires_grad = False
for param in model.layer4.parameters():
    param.requires_grad = True

model.fc = nn.Sequential(
    nn.Dropout(0.3),
    nn.Linear(2048, 512),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(512, 4)
)

# Training with PyTorch Lightning
class CorrosionModule(pl.LightningModule):
    def __init__(self, model, lr):
        super().__init__()
        self.model = model
        self.lr = lr
        self.criterion = nn.CrossEntropyLoss()

    def forward(self, x):
        return self.model(x)

    def training_step(self, batch, batch_idx):
        x, y = batch
        logits = self(x)
        loss = self.criterion(logits, y)
        acc = (logits.argmax(1) == y).float().mean()
        self.log('train_loss', loss)
        self.log('train_acc', acc)
        return loss

    def validation_step(self, batch, batch_idx):
        x, y = batch
        logits = self(x)
        loss = self.criterion(logits, y)
        acc = (logits.argmax(1) == y).float().mean()
        self.log('val_loss', loss)
        self.log('val_acc', acc)

    def configure_optimizers(self):
        return torch.optim.AdamW(self.parameters(), lr=self.lr)
```

```
    module = CorrosionModule(model, learning_rate)
    trainer = pl.Trainer(
        max_epochs=epochs,
        accelerator='gpu',
        devices=1,
        callbacks=[
            pl.callbacks.ModelCheckpoint(monitor='val_acc', mode='max'),
            pl.callbacks.EarlyStopping(monitor='val_acc', patience=10, mode='max')
        ]
    )
    trainer.fit(module, train_loader, val_loader)

    return model
```

---

7. RAG SYSTEM

7.1 Embedding Pipeline

```python
# models/rag/embeddings.py
from sentence_transformers import SentenceTransformer
import numpy as np
from typing import List
import torch

class DocumentEmbedder:
    """
    Document embedding using BGE-large model.

    Features:
    - 1024-dimensional embeddings
    - Optimized for retrieval
    - GPU acceleration
    """

    def __init__(
        self,
        model_name: str = 'BAAI/bge-large-en-v1.5',
        device: str = None,
        max_length: int = 512
    ):
        self.device = device or ('cuda' if torch.cuda.is_available() else 'cpu')
        self.model = SentenceTransformer(model_name, device=self.device)
        self.max_length = max_length

    def embed_documents(
        self,
        documents: List[str],
        batch_size: int = 32,
        show_progress: bool = True
    ) -> np.ndarray:
        """
        Embed list of documents.
```

```
        Args:
            documents: List of text documents
            batch_size: Batch size for encoding

        Returns:
            embeddings: (n_documents, 1024) array
        """
        embeddings = self.model.encode(
            documents,
            batch_size=batch_size,
            show_progress_bar=show_progress,
            normalize_embeddings=True,
            convert_to_numpy=True
        )
        return embeddings

    def embed_query(self, query: str) -> np.ndarray:
        """
        Embed a query with instruction prefix.

        BGE models benefit from instruction prefix for queries.
        """
        instruction = "Represent this question for retrieving relevant documents: "
        query_with_instruction = instruction + query

        embedding = self.model.encode(
            query_with_instruction,
            normalize_embeddings=True,
            convert_to_numpy=True
        )
        return embedding
```

7.2 RAG Pipeline

```
# models/rag/pipeline.py
from typing import List, Dict, Optional
import numpy as np
from pymilvus import Collection, connections
from sentence_transformers import CrossEncoder
from vllm import LLM, SamplingParams
import redis
import hashlib
import json
from loguru import logger

from .embeddings import DocumentEmbedder

class RAGPipeline:
    """
    Full RAG pipeline for Oil & Gas knowledge queries.

    Components:
    1. Query embedding (BGE-large)
```

```
    2. Vector search (Milvus)
    3. Reranking (Cross-encoder)
    4. Generation (Llama 3.1 70B via vLLM)
    """

    SYSTEM_PROMPT = """You are an expert Oil & Gas engineer assistant.
Your role is to help operators and engineers with technical questions about equipment, maintenance, and

Instructions:
1. Base your answers ONLY on the provided context
2. If the context doesn't contain enough information, clearly state this
3. Always cite sources using [Source: document_name] format
4. Be precise and technical but accessible
5. For safety-related questions, always emphasize caution and proper procedures

Context:
{context}

Equipment Status (if available):
{equipment_status}

Question: {question}

Answer:"""

    def __init__(self, config: dict):
        self.config = config

        # Embedding model
        logger.info("Loading embedding model...")
        self.embedder = DocumentEmbedder(
            model_name=config['embedding_model']
        )

        # Reranker
        logger.info("Loading reranker...")
        self.reranker = CrossEncoder(
            config['reranker_model'],
            max_length=512
        )

        # Milvus connection
        connections.connect(
            host=config['milvus_host'],
            port=config['milvus_port']
        )
        self.collection = Collection(config['collection_name'])
        self.collection.load()

        # LLM
        logger.info("Loading LLM...")
        self.llm = LLM(
            model=config['llm_model'],
            tensor_parallel_size=config.get('tensor_parallel_size', 4),
```

```
            gpu_memory_utilization=0.9
        )
        self.sampling_params = SamplingParams(
            temperature=config.get('temperature', 0.1),
            max_tokens=config.get('max_tokens', 1024),
            top_p=0.95
        )

        # Cache
        self.cache = redis.Redis(
            host=config.get('redis_host', 'localhost'),
            port=config.get('redis_port', 6379)
        )
        self.cache_ttl = config.get('cache_ttl', 3600)

    def retrieve(
        self,
        query: str,
        top_k: int = 20
    ) -> List[Dict]:
        """Retrieve relevant documents"""
        # Embed query
        query_embedding = self.embedder.embed_query(query)

        # Search Milvus
        results = self.collection.search(
            data=[query_embedding.tolist()],
            anns_field="embedding",
            param={
                "metric_type": "COSINE",
                "params": {"nprobe": 10}
            },
            limit=top_k,
            output_fields=["chunk_id", "document_id", "content", "metadata"]
        )

        chunks = []
        for hit in results[0]:
            chunks.append({
                'chunk_id': hit.entity.get('chunk_id'),
                'document_id': hit.entity.get('document_id'),
                'content': hit.entity.get('content'),
                'metadata': hit.entity.get('metadata'),
                'score': hit.score
            })

        return chunks

    def rerank(
        self,
        query: str,
        chunks: List[Dict],
        top_k: int = 5
    ) -> List[Dict]:
```

```
        """Rerank chunks using cross-encoder"""
        if not chunks:
            return []

        pairs = [(query, chunk['content']) for chunk in chunks]
        scores = self.reranker.predict(pairs)

        for i, score in enumerate(scores):
            chunks[i]['rerank_score'] = float(score)

        # Sort by rerank score
        chunks = sorted(chunks, key=lambda x: x['rerank_score'], reverse=True)

        return chunks[:top_k]

    def generate(
        self,
        query: str,
        context: str,
        equipment_status: str = ""
    ) -> str:
        """Generate response using LLM"""
        prompt = self.SYSTEM_PROMPT.format(
            context=context,
            equipment_status=equipment_status or "No specific equipment context provided.",
            question=query
        )

        outputs = self.llm.generate([prompt], self.sampling_params)
        response = outputs[0].outputs[0].text

        return response

    def query(
        self,
        query: str,
        equipment_context: Optional[Dict] = None,
        use_cache: bool = True
    ) -> Dict:
        """
        Full RAG query.

        Returns:
            {
                'query': original query,
                'response': generated answer,
                'sources': list of source documents,
                'latency_ms': total latency
            }
        """
        import time
        start = time.time()

        # Check cache
```

```
cache_key = f"rag:{hashlib.sha256(query.encode()).hexdigest()}"
if use_cache:
    cached = self.cache.get(cache_key)
    if cached:
        result = json.loads(cached)
        result['cached'] = True
        return result

# Retrieve
chunks = self.retrieve(query, top_k=20)

# Rerank
top_chunks = self.rerank(query, chunks, top_k=5)

# Format context
context_parts = []
for i, chunk in enumerate(top_chunks, 1):
    context_parts.append(f"[Document {i}]\n{chunk['content']}")
context = "\n\n---\n\n".join(context_parts)

# Format equipment status
equipment_status = ""
if equipment_context:
    equipment_status = json.dumps(equipment_context, indent=2)

# Generate
response = self.generate(query, context, equipment_status)

latency_ms = int((time.time() - start) * 1000)

result = {
    'query': query,
    'response': response,
    'sources': [
        {
            'document_id': c['document_id'],
            'content_preview': c['content'][:200],
            'score': c['rerank_score']
        }
        for c in top_chunks
    ],
    'latency_ms': latency_ms,
    'cached': False
}

# Cache result
if use_cache:
    self.cache.setex(cache_key, self.cache_ttl, json.dumps(result))

return result
```

---

8. MODEL TRAINING INFRASTRUCTURE

8.1 Distributed Training

```python
# training/distributed.py
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data.distributed import DistributedSampler
import os

def setup_distributed(rank: int, world_size: int):
    """Initialize distributed training"""
    os.environ['MASTER_ADDR'] = os.environ.get('MASTER_ADDR', 'localhost')
    os.environ['MASTER_PORT'] = os.environ.get('MASTER_PORT', '12355')

    dist.init_process_group(
        backend='nccl',
        rank=rank,
        world_size=world_size
    )
    torch.cuda.set_device(rank)

def cleanup_distributed():
    """Clean up distributed training"""
    dist.destroy_process_group()

def train_distributed(
    rank: int,
    world_size: int,
    model_class,
    config: dict
):
    """Distributed training function"""
    setup_distributed(rank, world_size)

    # Create model and move to GPU
    model = model_class(**config['model']).cuda(rank)
    model = DDP(model, device_ids=[rank])

    # Create distributed sampler
    train_sampler = DistributedSampler(
        train_dataset,
        num_replicas=world_size,
        rank=rank,
        shuffle=True
    )

    train_loader = DataLoader(
        train_dataset,
        batch_size=config['training']['batch_size'],
        sampler=train_sampler,
        num_workers=4,
        pin_memory=True
```

```
    )

    # Training loop
    optimizer = torch.optim.AdamW(model.parameters(), lr=config['training']['lr'])

    for epoch in range(config['training']['epochs']):
        train_sampler.set_epoch(epoch)
        model.train()

        for batch in train_loader:
            optimizer.zero_grad()
            loss = model(batch)
            loss.backward()
            optimizer.step()

        # Only save on rank 0
        if rank == 0:
            torch.save(model.module.state_dict(), f'model_epoch_{epoch}.pt')

    cleanup_distributed()
```

8.2 Training on Kubernetes

```yaml
# kubernetes/training-job.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: anomaly-detector-training
  namespace: oilgas-ml
spec:
  parallelism: 4  # Number of GPUs
  completions: 1
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: trainer
          image: harbor.oilgas.internal/oilgas/ml-trainer:latest
          command:
            - python
            - -m
            - torch.distributed.launch
            - --nproc_per_node=1
            - --nnodes=4
            - --node_rank=$(JOB_COMPLETION_INDEX)
            - --master_addr=anomaly-trainer-0.trainer
            - --master_port=12355
            - train.py
            - --config=/config/training_config.yaml
          resources:
            limits:
              nvidia.com/gpu: 1
              memory: 32Gi
              cpu: 8
```

```
            requests:
              nvidia.com/gpu: 1
              memory: 16Gi
              cpu: 4
          volumeMounts:
            - name: config
              mountPath: /config
            - name: data
              mountPath: /data
            - name: output
              mountPath: /output
          env:
            - name: MLFLOW_TRACKING_URI
              value: "http://mlflow.oilgas.internal:5000"
            - name: INFLUXDB_TOKEN
              valueFrom:
                secretKeyRef:
                  name: ml-secrets
                  key: influxdb-token
      volumes:
        - name: config
          configMap:
            name: training-config
        - name: data
          persistentVolumeClaim:
            claimName: training-data-pvc
        - name: output
          persistentVolumeClaim:
            claimName: training-output-pvc
      nodeSelector:
        nvidia.com/gpu.present: "true"
```

---

9. MODEL DEPLOYMENT

9.1 TensorRT Optimization

```python
# deployment/tensorrt_converter.py
import torch
import tensorrt as trt
import numpy as np
from pathlib import Path

class TensorRTConverter:
    """Convert PyTorch models to TensorRT for edge deployment"""

    def __init__(
        self,
        precision: str = 'fp16',  # 'fp32', 'fp16', 'int8'
        max_batch_size: int = 1,
        workspace_size: int = 1 << 30  # 1GB
    ):
        self.precision = precision
```

```
        self.max_batch_size = max_batch_size
        self.workspace_size = workspace_size

        self.logger = trt.Logger(trt.Logger.WARNING)

    def convert_onnx_to_trt(
        self,
        onnx_path: str,
        output_path: str,
        input_shapes: dict
    ):
        """Convert ONNX model to TensorRT engine"""
        builder = trt.Builder(self.logger)
        network = builder.create_network(
            1 << int(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH)
        )
        parser = trt.OnnxParser(network, self.logger)

        # Parse ONNX
        with open(onnx_path, 'rb') as f:
            if not parser.parse(f.read()):
                for error in range(parser.num_errors):
                    print(f"ONNX Parse Error: {parser.get_error(error)}")
                raise RuntimeError("Failed to parse ONNX model")

        # Configure builder
        config = builder.create_builder_config()
        config.max_workspace_size = self.workspace_size

        if self.precision == 'fp16':
            config.set_flag(trt.BuilderFlag.FP16)
        elif self.precision == 'int8':
            config.set_flag(trt.BuilderFlag.INT8)
            # TODO: Add calibrator for INT8

        # Optimization profiles for dynamic shapes
        profile = builder.create_optimization_profile()
        for name, shapes in input_shapes.items():
            profile.set_shape(
                name,
                min=shapes['min'],
                opt=shapes['opt'],
                max=shapes['max']
            )
        config.add_optimization_profile(profile)

        # Build engine
        engine = builder.build_engine(network, config)

        if engine is None:
            raise RuntimeError("Failed to build TensorRT engine")

        # Serialize
        with open(output_path, 'wb') as f:
```

```
            f.write(engine.serialize())

        print(f"TensorRT engine saved to {output_path}")
        return output_path

    @staticmethod
    def export_pytorch_to_onnx(
        model: torch.nn.Module,
        dummy_input: torch.Tensor,
        output_path: str,
        input_names: list = ['input'],
        output_names: list = ['output'],
        dynamic_axes: dict = None
    ):
        """Export PyTorch model to ONNX"""
        model.eval()

        torch.onnx.export(
            model,
            dummy_input,
            output_path,
            input_names=input_names,
            output_names=output_names,
            dynamic_axes=dynamic_axes or {
                'input': {0: 'batch_size'},
                'output': {0: 'batch_size'}
            },
            opset_version=17,
            do_constant_folding=True
        )

        print(f"ONNX model saved to {output_path}")
        return output_path
```

9.2 Edge Inference Service

```
# deployment/edge_inference.py
import numpy as np
import tensorrt as trt
import pycuda.driver as cuda
import pycuda.autoinit
from typing import Dict, Any
import json
from loguru import logger

class EdgeInferenceService:
    """
    TensorRT inference service for edge deployment.

    Optimized for NVIDIA Jetson AGX Orin.
    """

    def __init__(self, config_path: str):
        with open(config_path) as f:
```

```
            self.config = json.load(f)

        self.logger = trt.Logger(trt.Logger.WARNING)
        self.models = {}

        # Load models
        self._load_models()

    def _load_models(self):
        """Load TensorRT engines"""
        for name, model_config in self.config['models'].items():
            engine_path = model_config['engine_path']
            logger.info(f"Loading {name} from {engine_path}")

            with open(engine_path, 'rb') as f:
                runtime = trt.Runtime(self.logger)
                engine = runtime.deserialize_cuda_engine(f.read())

            context = engine.create_execution_context()

            # Allocate buffers
            inputs, outputs, bindings, stream = self._allocate_buffers(engine)

            self.models[name] = {
                'engine': engine,
                'context': context,
                'inputs': inputs,
                'outputs': outputs,
                'bindings': bindings,
                'stream': stream
            }

    def _allocate_buffers(self, engine):
        """Allocate GPU buffers"""
        inputs = []
        outputs = []
        bindings = []
        stream = cuda.Stream()

        for binding in engine:
            size = trt.volume(engine.get_binding_shape(binding))
            dtype = trt.nptype(engine.get_binding_dtype(binding))

            # Allocate host and device buffers
            host_mem = cuda.pagelocked_empty(size, dtype)
            device_mem = cuda.mem_alloc(host_mem.nbytes)

            bindings.append(int(device_mem))

            if engine.binding_is_input(binding):
                inputs.append({'host': host_mem, 'device': device_mem})
            else:
                outputs.append({'host': host_mem, 'device': device_mem})
```

```
        return inputs, outputs, bindings, stream

    def infer(
        self,
        model_name: str,
        input_data: np.ndarray
    ) -> np.ndarray:
        """Run inference"""
        model = self.models[model_name]

        # Copy input to device
        np.copyto(model['inputs'][0]['host'], input_data.ravel())
        cuda.memcpy_htod_async(
            model['inputs'][0]['device'],
            model['inputs'][0]['host'],
            model['stream']
        )

        # Run inference
        model['context'].execute_async_v2(
            bindings=model['bindings'],
            stream_handle=model['stream'].handle
        )

        # Copy output to host
        cuda.memcpy_dtoh_async(
            model['outputs'][0]['host'],
            model['outputs'][0]['device'],
            model['stream']
        )
        model['stream'].synchronize()

        return model['outputs'][0]['host'].copy()

    def predict_anomaly(self, features: np.ndarray) -> Dict[str, Any]:
        """Predict anomaly from vibration features"""
        # Normalize
        features = (features - self.config['anomaly']['mean']) / self.config['anomaly']['std']

        # Inference
        output = self.infer('anomaly_detector', features.astype(np.float32))

        # Calculate reconstruction error
        reconstruction = output.reshape(features.shape)
        error = np.mean((features - reconstruction)**2)

        threshold = self.config['anomaly']['threshold']
        is_anomaly = error > threshold

        return {
            'anomaly_score': float(error),
            'threshold': threshold,
            'is_anomaly': bool(is_anomaly),
            'reconstruction': reconstruction.tolist()
```

```
        }
```

---

10. MONITORING & RETRAINING

10.1 Model Performance Monitoring

```python
# monitoring/model_monitor.py
from prometheus_client import Counter, Histogram, Gauge
import numpy as np
from datetime import datetime
from typing import Dict

# Prometheus metrics
PREDICTION_COUNTER = Counter(
    'ml_predictions_total',
    'Total number of predictions',
    ['model', 'status']
)

PREDICTION_LATENCY = Histogram(
    'ml_prediction_latency_seconds',
    'Prediction latency',
    ['model'],
    buckets=[0.01, 0.025, 0.05, 0.075, 0.1, 0.25, 0.5, 1.0]
)

MODEL_ACCURACY = Gauge(
    'ml_model_accuracy',
    'Model accuracy (rolling)',
    ['model']
)

DATA_DRIFT = Gauge(
    'ml_data_drift_score',
    'Data drift detection score',
    ['model', 'feature']
)

class ModelMonitor:
    """Monitor model performance and data drift"""

    def __init__(self, config: dict):
        self.config = config
        self.baseline_stats = {}
        self.prediction_buffer = []
        self.buffer_size = config.get('buffer_size', 1000)

    def record_prediction(
        self,
        model_name: str,
        input_data: np.ndarray,
        prediction: np.ndarray,
```

```
    latency: float,
    ground_truth: np.ndarray = None
):
    """Record prediction for monitoring"""
    # Update counters
    PREDICTION_COUNTER.labels(model=model_name, status='success').inc()
    PREDICTION_LATENCY.labels(model=model_name).observe(latency)

    # Store in buffer
    self.prediction_buffer.append({
        'timestamp': datetime.utcnow(),
        'model': model_name,
        'input': input_data,
        'prediction': prediction,
        'ground_truth': ground_truth
    })

    # Trim buffer
    if len(self.prediction_buffer) > self.buffer_size:
        self.prediction_buffer = self.prediction_buffer[-self.buffer_size:]

    # Check data drift
    self._check_data_drift(model_name, input_data)

    # Update accuracy if ground truth available
    if ground_truth is not None:
        self._update_accuracy(model_name, prediction, ground_truth)

def _check_data_drift(self, model_name: str, input_data: np.ndarray):
    """Check for data drift using KS test"""
    from scipy import stats

    if model_name not in self.baseline_stats:
        return

    baseline = self.baseline_stats[model_name]

    for i, feature_name in enumerate(baseline['feature_names']):
        current = input_data[:, i] if input_data.ndim > 1 else [input_data[i]]

        # KS test
        statistic, p_value = stats.ks_2samp(
            baseline['distributions'][feature_name],
            current
        )

        DATA_DRIFT.labels(model=model_name, feature=feature_name).set(statistic)

        # Alert if significant drift
        if p_value < 0.01:
            self._alert_drift(model_name, feature_name, statistic)

def _update_accuracy(
    self,
```

```
        model_name: str,
        prediction: np.ndarray,
        ground_truth: np.ndarray
    ):
        """Update rolling accuracy"""
        correct = np.sum(prediction == ground_truth)
        total = len(ground_truth)

        # Get recent predictions with ground truth
        recent = [p for p in self.prediction_buffer[-100:]
                  if p['model'] == model_name and p['ground_truth'] is not None]

        if recent:
            all_correct = sum(
                np.sum(p['prediction'] == p['ground_truth'])
                for p in recent
            )
            all_total = sum(len(p['ground_truth']) for p in recent)
            accuracy = all_correct / all_total

            MODEL_ACCURACY.labels(model=model_name).set(accuracy)

    def set_baseline(self, model_name: str, data: np.ndarray, feature_names: list):
        """Set baseline distribution for drift detection"""
        self.baseline_stats[model_name] = {
            'feature_names': feature_names,
            'distributions': {
                name: data[:, i] for i, name in enumerate(feature_names)
            },
            'mean': np.mean(data, axis=0),
            'std': np.std(data, axis=0)
        }

    def _alert_drift(self, model_name: str, feature: str, score: float):
        """Alert on data drift"""
        # TODO: Send to alerting system
        print(f"DATA DRIFT ALERT: {model_name}/{feature} - score: {score:.4f}")
```

10.2 Automated Retraining

```
# retraining/trigger.py
from datetime import datetime, timedelta
from loguru import logger

class RetrainingTrigger:
    """Trigger model retraining based on conditions"""

    def __init__(self, config: dict):
        self.config = config
        self.last_retrain = {}

    def should_retrain(self, model_name: str, metrics: dict) -> bool:
        """Check if model should be retrained"""
        # Time-based trigger
```

```
        min_interval = self.config.get('min_retrain_interval_days', 7)
        last = self.last_retrain.get(model_name, datetime.min)
        if datetime.utcnow() - last < timedelta(days=min_interval):
            return False

        # Performance-based trigger
        accuracy_threshold = self.config.get('accuracy_threshold', 0.90)
        if metrics.get('accuracy', 1.0) < accuracy_threshold:
            logger.info(f"Retraining {model_name}: accuracy dropped below {accuracy_threshold}")
            return True

        # Drift-based trigger
        drift_threshold = self.config.get('drift_threshold', 0.2)
        if metrics.get('max_drift_score', 0.0) > drift_threshold:
            logger.info(f"Retraining {model_name}: data drift detected")
            return True

        # Data volume trigger
        new_samples = metrics.get('new_samples_since_last_train', 0)
        sample_threshold = self.config.get('sample_threshold', 10000)
        if new_samples >= sample_threshold:
            logger.info(f"Retraining {model_name}: {new_samples} new samples available")
            return True

        return False

    def trigger_retrain(self, model_name: str):
        """Trigger retraining job"""
        from kubernetes import client, config

        config.load_incluster_config()
        batch_v1 = client.BatchV1Api()

        job_name = f"{model_name}-retrain-{datetime.utcnow().strftime('%Y%m%d%H%M')}"

        job = client.V1Job(
            metadata=client.V1ObjectMeta(name=job_name),
            spec=client.V1JobSpec(
                template=client.V1PodTemplateSpec(
                    spec=client.V1PodSpec(
                        containers=[
                            client.V1Container(
                                name="trainer",
                                image="harbor.oilgas.internal/oilgas/ml-trainer:latest",
                                command=["python", "train.py", "--model", model_name],
                                resources=client.V1ResourceRequirements(
                                    limits={"nvidia.com/gpu": "1"}
                                )
                            )
                        ],
                        restart_policy="Never"
                    )
                )
            )
```

```
        )

        batch_v1.create_namespaced_job(namespace="oilgas-ml", body=job)
        logger.info(f"Triggered retraining job: {job_name}")

        self.last_retrain[model_name] = datetime.utcnow()
```

---

11. EXPERIMENT TRACKING

11.1 MLflow Integration

```
# utils/experiment_tracking.py
import mlflow
from mlflow.tracking import MlflowClient
from typing import Dict, Any
import json

class ExperimentTracker:
    """MLflow experiment tracking wrapper"""

    def __init__(self, tracking_uri: str, experiment_name: str):
        mlflow.set_tracking_uri(tracking_uri)
        mlflow.set_experiment(experiment_name)
        self.client = MlflowClient()

    def start_run(self, run_name: str, tags: Dict[str, str] = None):
        """Start new run"""
        return mlflow.start_run(run_name=run_name, tags=tags)

    def log_params(self, params: Dict[str, Any]):
        """Log parameters"""
        for key, value in params.items():
            if isinstance(value, dict):
                for k, v in value.items():
                    mlflow.log_param(f"{key}.{k}", v)
            else:
                mlflow.log_param(key, value)

    def log_metrics(self, metrics: Dict[str, float], step: int = None):
        """Log metrics"""
        mlflow.log_metrics(metrics, step=step)

    def log_model(
        self,
        model,
        artifact_path: str,
        registered_model_name: str = None
    ):
        """Log model"""
        mlflow.pytorch.log_model(
            model,
            artifact_path,
```

```
            registered_model_name=registered_model_name
        )

    def log_artifact(self, local_path: str, artifact_path: str = None):
        """Log artifact"""
        mlflow.log_artifact(local_path, artifact_path)

    def promote_model(
        self,
        model_name: str,
        version: int,
        stage: str  # 'Staging', 'Production', 'Archived'
    ):
        """Promote model to stage"""
        self.client.transition_model_version_stage(
            name=model_name,
            version=version,
            stage=stage
        )

    def get_best_run(
        self,
        experiment_name: str,
        metric: str,
        maximize: bool = True
    ):
        """Get best run by metric"""
        experiment = self.client.get_experiment_by_name(experiment_name)
        runs = self.client.search_runs(
            experiment_ids=[experiment.experiment_id],
            order_by=[f"metrics.{metric} {'DESC' if maximize else 'ASC'}"],
            max_results=1
        )
        return runs[0] if runs else None
```

---

12. BEST PRACTICES

12.1 Model Development Checklist

- ☐ Define clear problem statement and success metrics
- ☐ Exploratory data analysis completed
- ☐ Data quality checks implemented
- ☐ Baseline model established
- ☐ Cross-validation strategy defined
- ☐ Hyperparameter tuning performed
- ☐ Model interpretability addressed
- ☐ Performance on edge cases tested
- ☐ A/B testing plan created
- ☐ Monitoring metrics defined
- ☐ Rollback procedure documented

12.2 Code Quality Standards

```python
# Example of well-documented model class
class AnomalyDetector:
    """
    Autoencoder-based anomaly detector for vibration data.

    This model learns to reconstruct normal vibration patterns. Anomalies
    are detected when reconstruction error exceeds a learned threshold.

    Attributes:
        model: PyTorch autoencoder model
        threshold: Anomaly detection threshold (2-sigma)
        scaler: Feature normalization scaler

    Example:
        >>> detector = AnomalyDetector.load('models/anomaly_v1.0')
        >>> features = extract_features(vibration_data)
        >>> result = detector.predict(features)
        >>> print(f"Anomaly: {result['is_anomaly']}, Score: {result['score']:.4f}")

    References:
        - Architecture based on: "Deep Learning for Anomaly Detection" (Chalapathy, 2019)
        - Threshold selection: "Unsupervised Anomaly Detection" (Goldstein, 2016)
    """

    def __init__(self, config: dict):
        """
        Initialize anomaly detector.

        Args:
            config: Configuration dictionary with keys:
                - model_path: Path to trained model weights
                - threshold: Anomaly threshold (default: auto-calculated)
                - device: Compute device ('cuda' or 'cpu')

        Raises:
            FileNotFoundError: If model_path doesn't exist
            ValueError: If threshold is negative
        """
        pass
```

12.3 Performance Optimization Tips

1. **Data Loading**: Use `num_workers > 0` in DataLoader, enable `pin_memory=True`
2. **Mixed Precision**: Use `torch.cuda.amp` for FP16 training
3. **Gradient Accumulation**: For large effective batch sizes
4. **Model Compilation**: Use `torch.compile()` for PyTorch 2.0+
5. **TensorRT**: Export to TensorRT for production inference
6. **Batch Inference**: Always batch predictions when possible
7. **Caching**: Cache embeddings and frequent computations

END OF ML ENGINEER GUIDE